



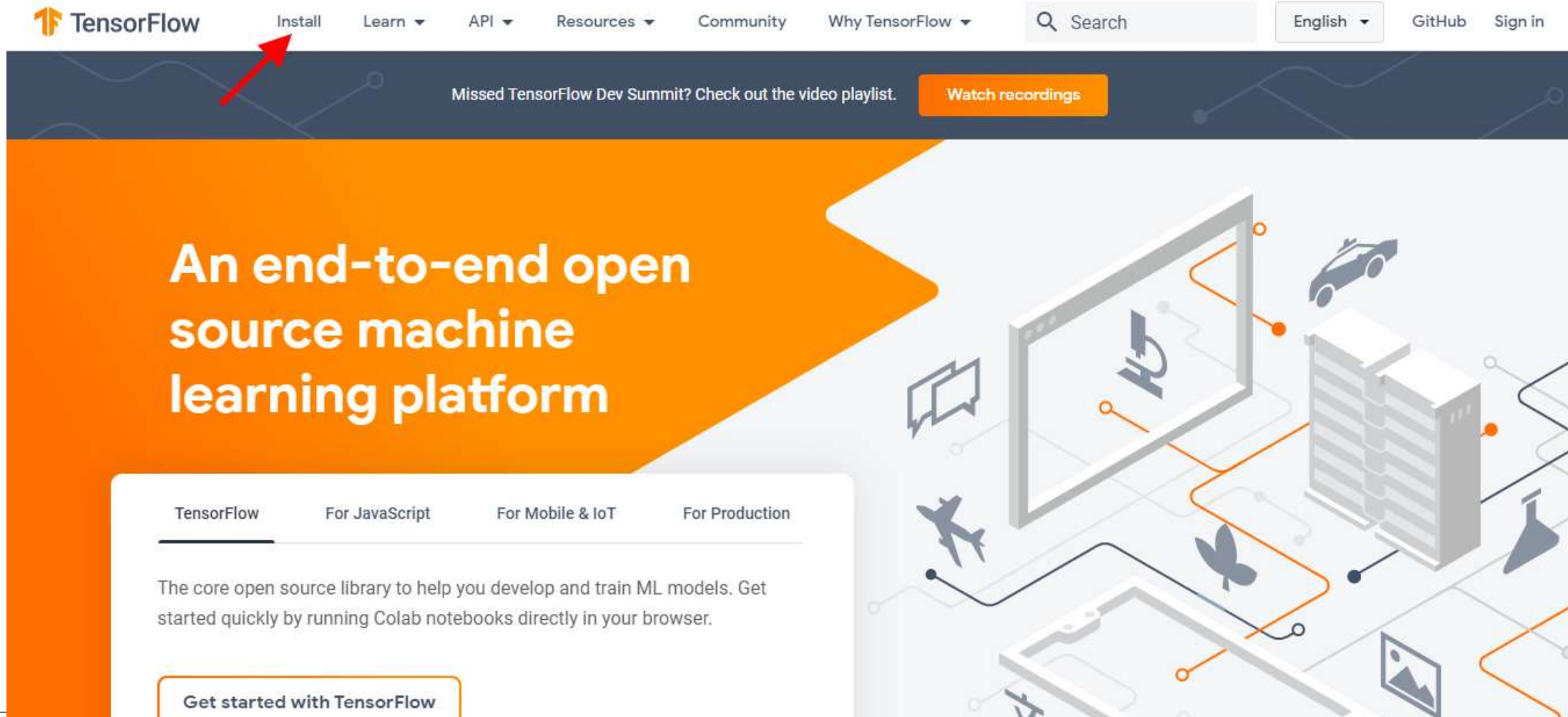
TENSORFLOW

Deep Learning

Installation of Tensorflow

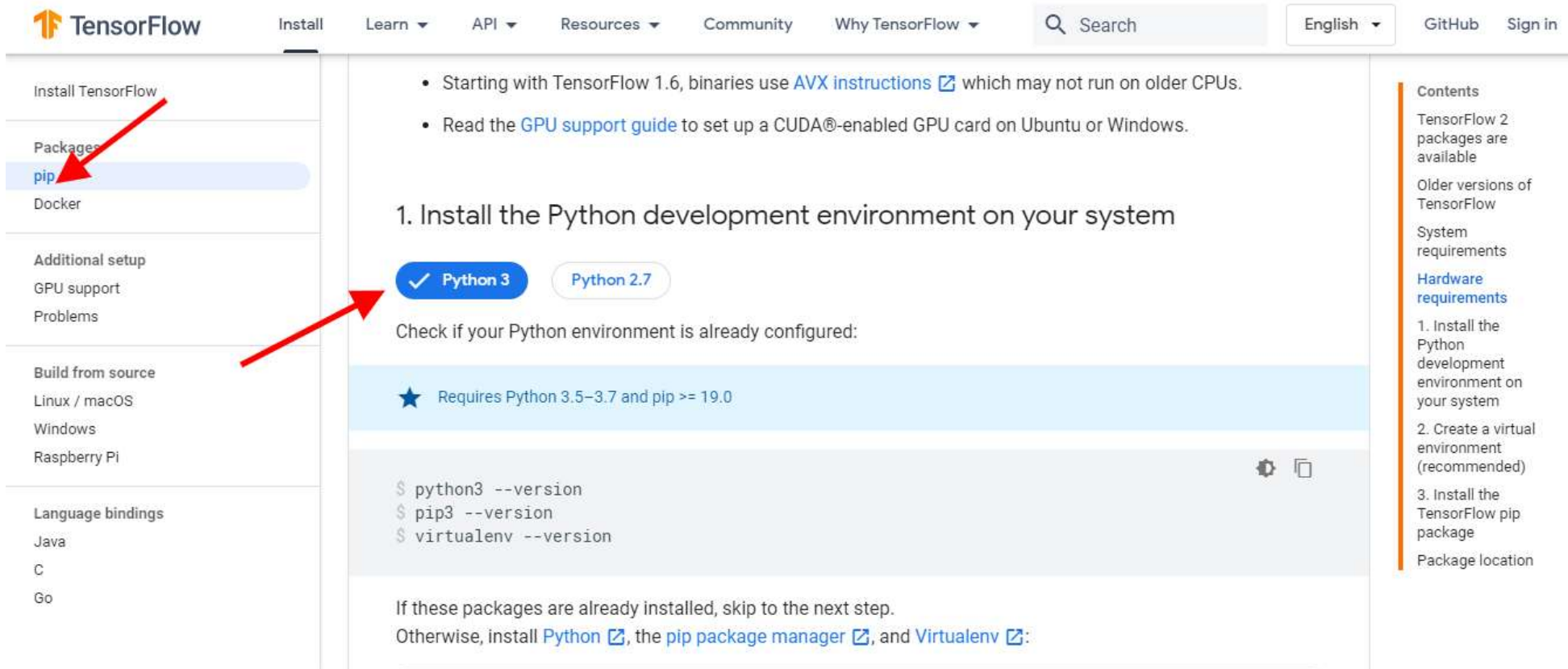
- Tensorflow is a free and open-source software library used to do computational mathematics to build machine learning models more profoundly deep learning models.
- It is a product of Google built by Google's brain team, hence it provides a vast range of operations performance with ease that is compatible with human behavior for ML and DL. It can run on multiple CPUs and GPUs.

Step 1: Click on Install on top navigation bar of TensorFlow website.



Step 2: Get the python environment

- Before proceeding we need to get python environment. Choose pip in the left side and go to python section and install python environment to work on it.



The screenshot shows the TensorFlow website's installation page. The left sidebar contains a navigation menu with the following items: 'Install TensorFlow', 'Packages' (highlighted with a red arrow pointing to 'pip'), 'Additional setup' (containing 'GPU support' and 'Problems'), 'Build from source' (containing 'Linux / macOS', 'Windows', and 'Raspberry Pi'), and 'Language bindings' (containing 'Java', 'C', and 'Go'). The main content area is titled '1. Install the Python development environment on your system'. It features two tabs: 'Python 3' (selected with a checkmark and highlighted by a red arrow) and 'Python 2.7'. Below the tabs, it says 'Check if your Python environment is already configured:'. A light blue box contains a star icon and the text 'Requires Python 3.5–3.7 and pip >= 19.0'. Below this, a code block shows the commands: `$ python3 --version`, `$ pip3 --version`, and `$ virtualenv --version`. At the bottom, it states: 'If these packages are already installed, skip to the next step. Otherwise, install [Python](#), the [pip package manager](#), and [Virtualenv](#).' The right sidebar contains a 'Contents' section with links to 'TensorFlow 2 packages are available', 'Older versions of TensorFlow', 'System requirements', 'Hardware requirements', and a numbered list of steps: '1. Install the Python development environment on your system', '2. Create a virtual environment (recommended)', '3. Install the TensorFlow pip package', and 'Package location'.

TensorFlow

Install Learn API Resources Community Why TensorFlow Search English GitHub Sign in

Install TensorFlow

Packages

pip

Docker

Additional setup

GPU support

Problems

Build from source

Linux / macOS

Windows

Raspberry Pi

Language bindings

Java

C

Go

- Starting with TensorFlow 1.6, binaries use [AVX instructions](#) which may not run on older CPUs.
- Read the [GPU support guide](#) to set up a CUDA®-enabled GPU card on Ubuntu or Windows.

1. Install the Python development environment on your system

✓ Python 3 Python 2.7

Check if your Python environment is already configured:

★ Requires Python 3.5–3.7 and pip >= 19.0

```
$ python3 --version
$ pip3 --version
$ virtualenv --version
```

If these packages are already installed, skip to the next step.
Otherwise, install [Python](#), the [pip package manager](#), and [Virtualenv](#):

Contents

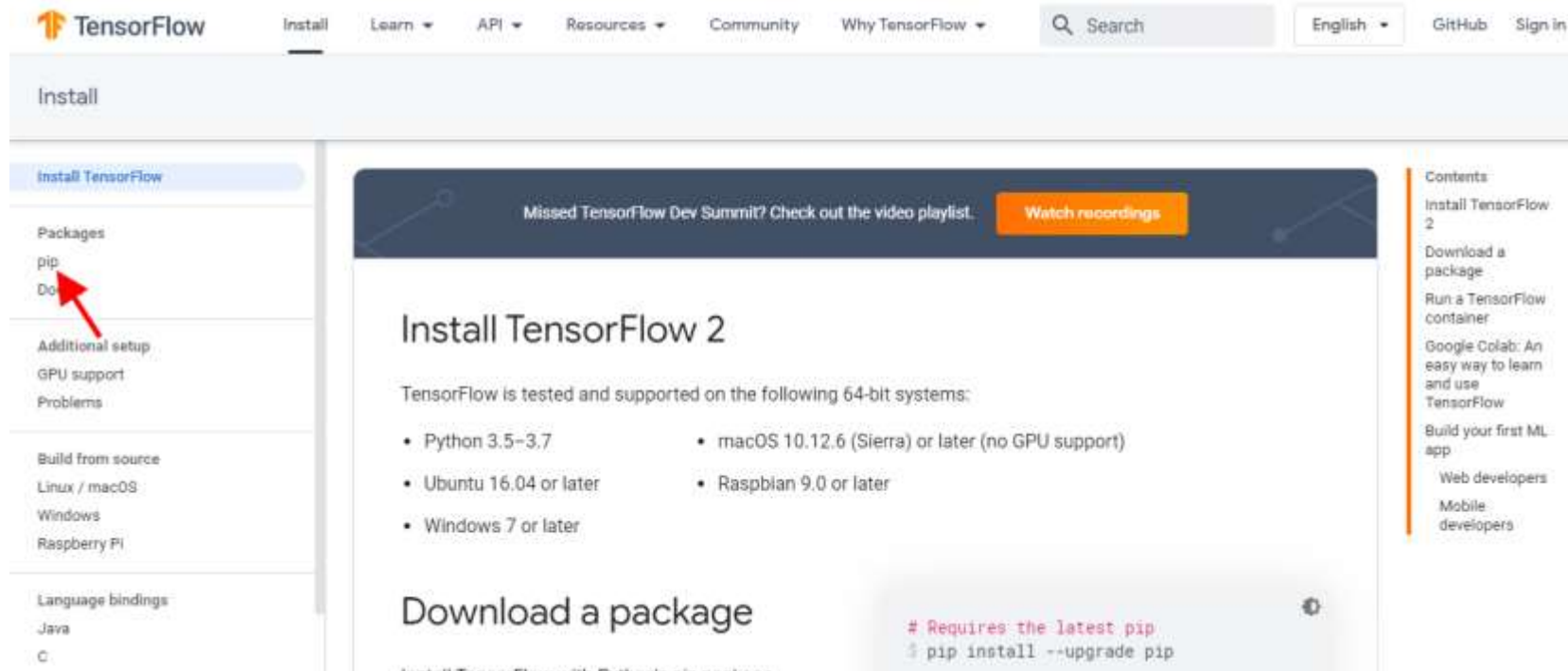
- [TensorFlow 2 packages are available](#)
- [Older versions of TensorFlow](#)
- [System requirements](#)
- [Hardware requirements](#)
- 1. Install the Python development environment on your system
- 2. Create a virtual environment (recommended)
- 3. Install the TensorFlow pip package
- [Package location](#)

Step 3: Python environment

- Python environment can be downloaded from python.org.

Step 4: pip

- To install Tensorflow, we need pip for python. Usually, pip comes already installed if the python version is 3 or above.
- For all OS users check the pip section of TensorFlow as shown in the following screenshot:



Step 5: Creating a Tensorflow virtual environment

2. Create a virtual environment (recommended)

Python virtual environments are used to isolate package installation from the system.

Ubuntu / mac OS Windows Conda

Create a new virtual environment by choosing a Python interpreter and making a `.\venv` directory to hold it:

```
C:\> virtualenv --system-site-packages -p python3 ./venv
```

Activate the virtual environment:

```
(venv) C:\> .\venv\Scripts\activate
```

```
pip install --user virtualenv
```

pytorch vs tensorflow vs keras

Parameter	Keras	TensorFlow	PyTorch
Performance and Speed	The constrain imposed by the moderate accomplishments with building bricks standardization.	In contrast, the graphics processing unit is optimized for running the many graph-based calculations.	Increased speed with operate the dynamic graph command is very important.
Ease of Use and Learning Curve	User-friendly interface	Steep learning curve	Simple to use, the glove provides many real-time graph relation growing.
Community and Ecosystem	Integrated ecosystem with TensorFlow	A robust community consensus and also adequate documentation concerning cybersecurity role should be provided.	The community's expansion on the base of the research is the key.
Deployment and Production Readiness	Gliding through the transition process from the prototyping into the deployment is an assurance for the success.	Ease of use and also enlarged capabilities with the accurate tools and also also rich libraries.	Inferior to TF in terms of the availability for the deplo

	TensorFlow	PyTorch	Keras
API Level	Both (High and Low)	Low	High
Architecture	Not easy to use	Complex, less readable	Simple, concise, readable
Datasets	Large datasets, high-performance	Large datasets, high-performance	Smaller datasets
Debugging	Difficult to conduct debugging	Good debugging capabilities	Simple network, so debugging is not often needed
Pretrained models?	Yes	Yes	Yes
Popularity	Second most popular of the three	Third most popular of the three	Most popular of the three
Speed	Fast, high-performance	Fast, high-performance	Slow, low performance
Written In	C++, CUDA, Python	Lua	Python

Neural network for handwritten digit classification

◦ Step 1: Import the Necessary Libraries

```
import tensorflow as tf  
  
from tensorflow.keras import datasets, layers, models  
  
import matplotlib.pyplot as plt  
  
# Load the MNIST dataset  
  
(train_images, train_labels), (test_images, test_labels) =  
datasets.mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz  
11490434/11490434 ————— 0s 0us/step
```

Neural network for handwritten digit classification

- Step 2: Preprocess the Data

```
# Normalize the images to the range of [0, 1]
```

```
train_images = train_images / 255.0
```

```
test_images = test_images / 255.0
```

Neural network for handwritten digit classification

◦ Step 3: Build the Neural Network Model

```
model = models.Sequential([  
    layers.Flatten(input_shape=(28, 28)),    # Flatten the  
    28x28 images into a 784-dimensional vector  
    layers.Dense(128, activation='relu'),    # Fully  
    connected layer with 128 units and ReLU activation  
    layers.Dense(10, activation='softmax')   # Output layer  
    with 10 units for the 10 classes  
])
```

Neural network for handwritten digit classification

- Step 4: Compile the Model

```
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

Neural network for handwritten digit classification

- Step 5: Train the Model

```
model.fit(train_images, train_labels, epochs=5)
```

```
Epoch 1/5  
1875/1875 ————— 8s 4ms/step - accuracy: 0.8793 - loss: 0.4206  
Epoch 2/5  
1875/1875 ————— 11s 4ms/step - accuracy: 0.9671 - loss: 0.1174  
Epoch 3/5  
1875/1875 ————— 9s 3ms/step - accuracy: 0.9783 - loss: 0.0736  
Epoch 4/5  
1875/1875 ————— 10s 3ms/step - accuracy: 0.9830 - loss: 0.0569  
Epoch 5/5  
1875/1875 ————— 10s 3ms/step - accuracy: 0.9868 - loss: 0.0420  
<keras.src.callbacks.history.History at 0x797dbe438b50>
```

Neural network for handwritten digit classification

- Step 6: Evaluate the Model

```
test_loss, test_acc = model.evaluate(test_images,  
test_labels)
```

```
print(f'Test accuracy: {test_acc}')
```

```
313/313 ————— 1s 2ms/step - accuracy: 0.9733 - loss: 0.0860
```

```
Test accuracy: 0.9768999814987183
```

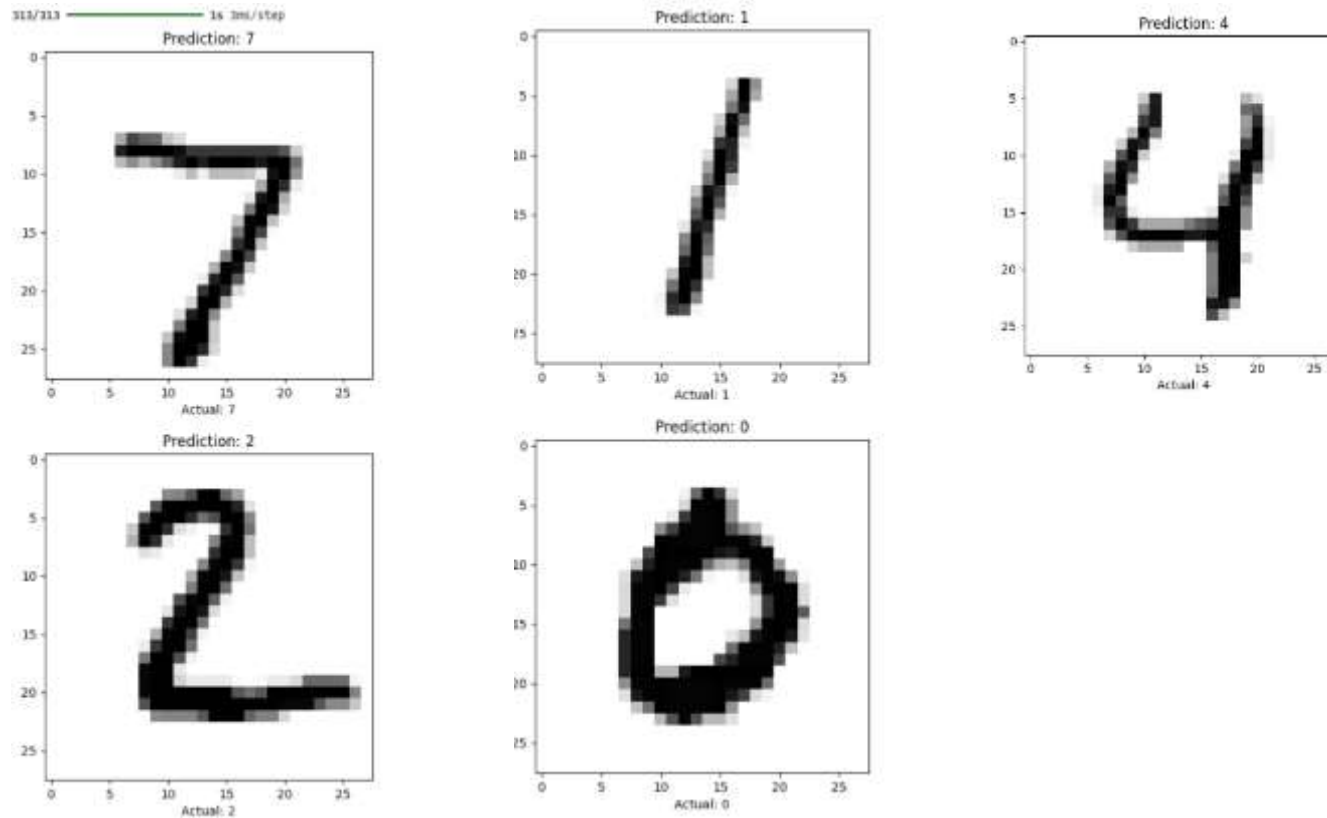
Neural network for handwritten digit classification

◦ Step 7: Make Predictions

```
predictions = model.predict(test_images)
# Display the first 5 predictions
for i in range(5):
    plt.grid(False)
    plt.imshow(test_images[i], cmap=plt.cm.binary)
    plt.xlabel(f'Actual: {test_labels[i]}')
    plt.title(f'Prediction: {tf.argmax(predictions[i])}')
    plt.show()
```

Neural network for handwritten digit classification

Step 7: Make Predictions





THANK YOU!!